



PROG7311

Part 1

Emeris

Rocston John Oppelt
ST10276665

Table of Contents

Introduction 2

TOGAF ADM cycle (Phases A-D) 2

 Phase A: Architecture Vision..... 2

 Phase B: Business Architecture 2

 Phase C: (Information Systems Architecture)..... 3

 Phase D: Technology Architecture..... 3

Design Pattern Selection 4

 State Pattern 4

 Strategy Pattern 5

 Observer Pattern 5

Non-Functional Requirements and Scalability 6

 Scalability Strategy 6

 Horizontal scaling..... 6

Bibliography 7

Introduction

TechMove is a logistics operation through the development of the Global Logistics Management System (GLMS). We will be replacing the outdated legacy system due to certain flaws such as inefficiency in delivery bookings, limited real-time shipment tracking and poor scalability which will impact customer satisfaction and operational performance in a negative way.

TOGAF ADM cycle (Phases A-D)

TOGAF (The Open Group Architecture Framework) was selected as the enterprise architecture framework for the Global Logistics due the fact it gives a structured, proven, and interactive methodology that aligns business processes with IT capabilities. Seeing that we are in the coordination industry where complex stakeholders' coordination, real time operations, scalability, and integration with external services such as a GPS and APIs are critical. This approach will reduce risks, improve maintainability of the code, and support future growths when the volume of contracts increases.

Phase A: Architecture Vision

The architecture vision phase is responsible for defining the purpose of the GLMS (Global Logistics Management System). The System is designed to enable customers to book deliveries, track shipments and manage transport efficiently. During this phase we determine the key stakeholders such as the customers, administrators, and drivers. We will also define the business requirements and constraints. Once all of this is visualized, we can present the plan and wait on approval to proceed to the next phase [1].

Phase B: Business Architecture

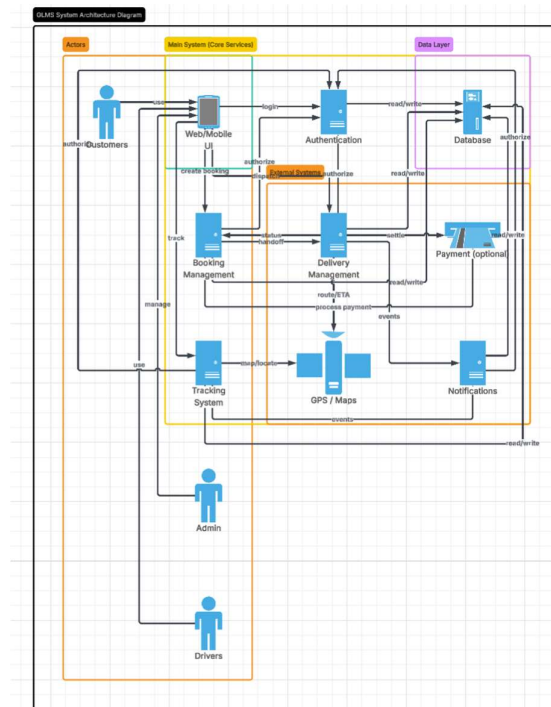
The business Architecture phase focuses on defining how the organization will operate the GLMS and how user will interact with the system. This means we will identify the key business processes such as booking deliveries, managing transport operations, and tracking shipments. Different user roles such as the customer, administrators and drivers are considered to ensure that the system supports their specific needs. The current business process is analysed and compared with the proposed system to identify improvements and enhance operational efficiency [2].

Phase C: (Information Systems Architecture)

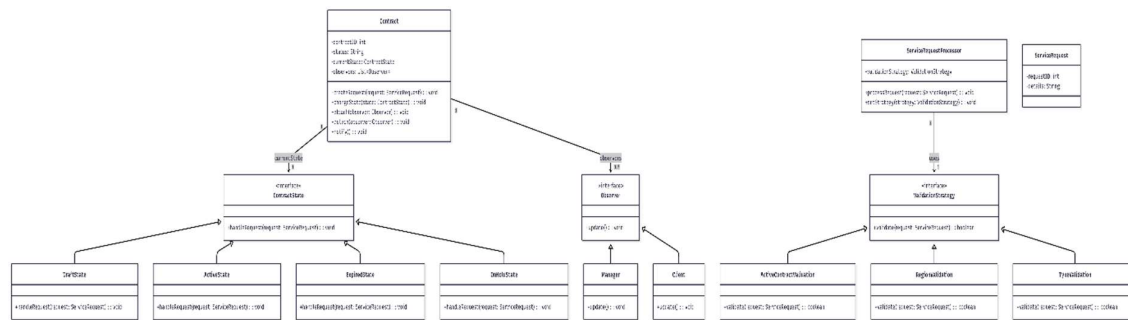
The information Systems Architecture defines the data and applications components of the GLMS. The system will include a user interface, backend services for the booking and tracking and a database to store user, delivery, and transaction data. APIs will be used to integrate external services such as the GPS tracking and notification systems.

Phase D: Technology Architecture

The technology Architecture outlines the infrastructure that is required to support the GLMS. The system will be deployed on cloud platforms to ensure scalability and availability. It will use web technologies for the frontend, backend API for the business logic and a relational database for data storage. External integrations such as GPS services and notification systems will be supported through secure APIs.



Design Pattern Selection



I will be using the following three design patterns:

- State Pattern
- Strategy Pattern
- Observer Pattern

State Pattern

Contracts will have different statuses such as Draft, Active, Expired and On Hold. Each status determines what operations are allowed. Without the use of a proper pattern, we would need to implement a lot of if-else statements which makes the code harder to maintain.

To solve this, the state pattern is used to create `ContractState ()` with a method like `handleRequest ()`. We will also implement the statuses such as `DraftState`, `ActiveState`, `ExpiredState` and `OnHoldState`. The contract statuses will allow us to determine how the contract is able to behave.

This fixes GLMS problems as we can decouple from the contract class which makes every clean and make the code maintainable. If we decide to add new states, we don't must change any code and the rules of the business are automatically enforced by the state classes [3].

Strategy Pattern

Service requests need different validation rules depending on contract type, the region and other business rules there might be. Hard coding this would lead to us violating open/closed principle.

We will define a `ValidationStrategy` interface with a method `validate ()` alongside with the strategies `ActiveContractValidation`, `RegionValidation` and `TypeValidation`. The `ServiceRequestProcessor` will hold a reference to a `ValidationStrategy` and use it dynamically.

This suits the GLMS as it will decouple validation logic from request processing, make it all flexible as new validation rules can be added without changing the existing code and support multiple contract types and scenarios which falls inline with the GLMS requirements [4].

Observer Pattern

Managers and clients will need real time updates when contracts change status or when a new request will be made. If we hard code the notifications, it will be inefficient and create tight coupling.

What we will do is make it that contract acts as a subject, the managers, and clients function as observers. When anything in the contract changes, `Contract.notify()` will update the observers automatically.

This leads to loose coupling which will make it that contracts do not need to know how many observers exist. Support multiple notification [5].

Non-Functional Requirements and Scalability

The GLMS needs to maintain high availability to ensure continuous access for customers booking deliveries, drivers updating shipment status and administrators managing the operations. The target availability will be set to 99.9% uptime. This will be achieved through cloud development with redundant infrastructure, automatic failover mechanisms, and multi-zone/ multi-region deployment where feasible.

Interoperability is important for the GLMS as it must seamlessly integrate with external systems like the GPS tracking provider, third-party notification services, payment gateways and new partners for the logistics platform. This will be accomplished using standardised RESTful APIs, JSON data formats, and secure authentication protocols (such as OAuth 2.0). These design choices ensure smooth data exchange and future integration with additional services without major system changes.

Scalability Strategy

In an event that TechMove has a competitor and experiences a sudden surge of 50,000 new contracts overnight, the GLMS architecture is specifically designed to handle the largely increased amount of load without the performance degrading.

The main strategy is horizontal scaling. Instead of upgrading the capacity of a single server, the system will automatically add multiple identical instances of application servers and database replicas, this way it provides better fault tolerance, near-unlimited growth potential and cost efficiency through cloud auto-scaling groups.

Horizontal scaling

For better horizontal scaling, these methods will come into play:

Load Balancing: An application load balancer will be placed in front of the web and API servers. It distributes incoming traffic such as delivery bookings, shipment tracking queries and notifications evenly across all active server instances. This ensures that one server does not get too overwhelmed and consistent response times even under high loads

Caching: Data that is commonly used such as contract statuses, shipment tracking and user sessions. These will be stored in an in-memory cache such as Redis. This reduces the load of the main database during peak times and will improve overall performance responsiveness.

Bibliography

- [1] TOGAF, "Preliminary Phase: Framework and Principles," [Online]. Available: http://www.togaf.com/admref/_chap05.html. [Accessed 26th March 2026].
- [2] Visual Paradigm, "Comprehensive Guide to Phase B: Business Architecture in TOGAF ADM," Visual Paradigm, [Online]. Available: <https://togaf.visual-paradigm.com/2025/01/20/comprehensive-guide-to-phase-b-business-architecture-in-togaf-adm/>.
- [3] GeekforGeek, "State Design Pattern," 13 Feb 2026. [Online]. Available: <https://www.geeksforgeeks.org/system-design/state-design-pattern/>.
- [4] GeekforGeek, "Strategy Design Pattern," 13 Feb 2026. [Online]. Available: <https://www.geeksforgeeks.org/system-design/strategy-pattern-set-1/>.
- [5] "Observer Design Pattern," 13 Feb 2026. [Online]. Available: <https://www.geeksforgeeks.org/system-design/observer-pattern-set-1-introduction/>.